Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

Catering Business Management System Database Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This document presents the progress and implementation of Activity 4: Database Users, Roles, Privileges, and Access Control for the Group08 Catering Business Management System.

The activity focuses on creating database roles, assigning appropriate privileges, and testing access restrictions using PostgreSQL. The goal is to apply Role-Based Access Control (RBAC) and the principle of least privilege, where each database user receives only the permissions needed for their assigned responsibilities.

2. Project Overview

The project used for this activity is the Catering Business Management System, which uses a PostgreSQL database named grp08_catering_db.

The system is designed to manage catering business operations, including customers, packages, menu items, reservations, payments, inventory, employees, event assignments, and cancellations.

The project uses Node.js and Express.js for its backend and PostgreSQL for database management.

Item	Details
Project	Catering Business Management System
Group	Group08
Database	grp08_catering_db

DBMS	PostgreSQL 18.6
Backend	Node.js and Express.js
Database Driver	node-postgres (pg)
Schema	public
Frontend	HTML, CSS, and JavaScript
Tables	11
Identity Sequences	9
Seed Records	31

A separate database named grp08_catering_db was used for Activity 4 testing to keep the access-control activities isolated from the original project database.

3. Database Inspection

The database schema and seed data were imported successfully.

During the initial inspection, the database contained 11 tables and 9 identity sequences. The tables and sequences were initially owned by the PostgreSQL postgres account.

Initial Record Counts

Table	Initial Records
customer	3
package	3
menu_item	5
employee	3
inventory	5
reservation	3

payment	2
reservation_menu	3
event_assignment	3
cancellation	1
admin	0
Total	**31**

All tables and sequences were initially owned by the PostgreSQL postgres account.

The inspection established the initial database structure and record counts before implementing the access-control configuration.

4. Role Creation and Configuration

Four NOLOGIN group roles and four LOGIN accounts were created.

4.1 Group Roles

Group Role	Purpose
grp08_catering_admin_grp	For database administration tasks
grp08_catering_staff_grp	For catering business operations
grp08_catering_customer_grp	For customer portal operations
grp08_catering_readonly_grp	For users who only need to view approved data

All group roles were created with the NOLOGIN attribute. This means they cannot be used directly to connect to the database.

4.2 Login Roles

Login Role	Assigned Group Role
grp08_catering_admin	grp08_catering_admin_grp
grp08_catering_staff	grp08_catering_staff_grp
grp08_catering_customer	grp08_catering_customer_grp
grp08_catering_auditor	grp08_catering_readonly_grp

- The login roles were created with the LOGIN attribute and assigned to their corresponding group roles.
- Passwords were configured locally and were not included in the SQL script or repository.
- The role configuration was checked to verify that the accounts did not have SUPERUSER, CREATEROLE, or CREATEDB privileges.

5. Privilege Assignment

Privileges were assigned according to each role's responsibility and the principle of least privilege.

5.1 Database and Schema Privileges

The four group roles were granted CONNECT permission on grp08_catering_db and USAGE permission on the public schema.

To restrict database access, the CONNECT privilege was revoked from PUBLIC on grp08_catering_db. Explicit CONNECT privileges were granted to the four intended group roles.

The CREATE privilege was also revoked from PUBLIC on the public schema. The group roles were granted USAGE permission, but CREATE permission was not granted.

Existing PUBLIC privileges on tables and sequences in the public schema were also revoked.

5.2 Administrative Role

The grp08_catering_admin_grp role was granted:

- SELECT
- INSERT
- UPDATE
- DELETE

These privileges apply to all existing tables in the public schema.

The administrative group was also granted ALL privileges on all existing database sequences.

However, the administrative role was not granted PostgreSQL SUPERUSER, CREATEROLE, or CREATEDB privileges. It was also not granted CREATE permission on the public schema.

5.3 Staff Role

- The grp08_catering_staff_grp role was granted:
  - SELECT
  - INSERT
  - UPDATE
- These privileges apply to the following operational tables:
  - cancellation
  - employee
  - event_assignment
  - inventory
  - menu_item
  - package
  - payment
  - reservation
  - reservation_menu
- The staff role also received limited customer column privileges:
  - SELECT on customer_id, first_name, last_name, email, phone, and address.
  - INSERT on first_name, last_name, email, phone, and address.
  - UPDATE on customer_id, first_name, last_name, email, phone, and address.
- The staff role received USAGE and SELECT privileges on eight assigned sequences:
  - cancellation_cancellation_id_seq
  - customer_customer_id_seq
  - employee_employee_id_seq
  - inventory_inventory_id_seq
  - menu_item_menu_item_id_seq

  - package_package_id_seq
  - payment_payment_id_seq
  - reservation_reservation_id_seq
- The following restrictions were applied:
  - No DELETE permission.
  - No access to the admin table.
  - No SELECT, INSERT, or UPDATE access to customer password_hash.
  - No CREATE permission on the public schema.

5.4 Customer Role

The grp08_catering_customer_grp role was granted the following permissions:

- SELECT access to permitted customer profile columns.
- INSERT access for customer registration.
- UPDATE access to permitted profile fields.
- SELECT access to packages and menu items.
- SELECT and INSERT access to reservations, reservation menu entries, payments, and cancellations.

The customer role received USAGE and SELECT privileges on four assigned sequences:

- customer_customer_id_seq
- reservation_reservation_id_seq
- payment_payment_id_seq
- cancellation_cancellation_id_seq

The following restrictions were applied:

- No SELECT access to customer password_hash.
- No UPDATE access to email
- No DELETE permission.
- No access to administrative records.
- No CREATE permission on the public schema.

5.5 Read-Only Role

- The grp08_catering_readonly_grp role was granted SELECT permission on three tables:
  - inventory
  - menu_item
  - package
- The auditor role was not granted INSERT, UPDATE, or DELETE privileges.
- It also has no access to customer information, reservation records, payment records, or administrative records.

- No sequence privileges were assigned to the read-only group.

6. Access Control Testing

The four login roles were tested separately to verify that their assigned privileges worked as expected.

6.1 Customer Account — grp08_catering_customer

The following tests were performed:

Operation	Result
Login and connect to database	Successful
View package and menu catalogs	Successful
View permitted profile columns	Successful
INSERT customer record	Successful
UPDATE permitted profile field	Successful
DELETE reservation	Denied
SELECT password_hash	Denied
UPDATE restricted email field	Denied
Access admin table	Denied

The customer account successfully performed its permitted database operations. Attempts to access restricted information or perform unauthorized modifications returned permission-denied errors.

6.2 Staff Account — grp08_catering_staff

The following tests were performed:

Operation	Result
Login and connect to database	Successful
SELECT reservation records	Successful
UPDATE reservation	Successful
DELETE reservation	Denied
Access admin table	Denied
SELECT customer password_hash	Denied
View permitted customer profile columns	Successful

The tests confirmed that the staff account could access and update permitted operational records but could not perform DELETE operations or access restricted administrative and customer information.

6.3 Read-Only Account — grp08_catering_auditor

The following tests were performed:

Operation	Result
Login and connect to database	Successful
SELECT inventory	Successful
SELECT menu_item	Successful
SELECT package	Successful
SELECT reservations	Denied
Access customer records	Denied

INSERT inventory record	Denied

The auditor account successfully retrieved the approved records but could not access restricted tables or perform database modifications.

6.4 Administrative Account — grp08_catering_admin

The following tests were performed:

Operation	Result
Connect to database	Successful
SELECT customer records	Successful
SELECT reservation records	Successful
SELECT admin records	Successful
INSERT inventory record	Successful
UPDATE inventory record	Successful
DELETE inventory record	Successful
CREATE table in public schema	Denied

The administrator account successfully performed the assigned table operations. However, its CREATE TABLE attempt was denied because it was not granted CREATE permission on the public schema.

The administrative role has table and sequence privileges without unrestricted PostgreSQL administrative privileges.

6.5 Testing Summary

The tests confirmed the following:

- The administrative account has the assigned table and sequence privileges.
- The staff account can perform permitted operational operations but cannot DELETE records.

- The customer account can perform permitted customer operations but cannot access restricted information.
- The auditor account can only read approved tables and cannot modify database records.
- The four login accounts have the intended group role memberships.
- Unauthorized access attempts were rejected by PostgreSQL.

Write operations tested inside transactions were rolled back afterward to avoid permanently modifying the original sample records.

6.6 Application Connection Testing

The Node.js backend was tested separately from the direct PostgreSQL role tests.

The following checks were completed:

- The database connection test returned Database connected successfully!.
- The connected database was identified as grp08_catering_db.
- The Express backend successfully started at http://localhost:5000.
- The /api/db-test endpoint returned a successful database connection response.
- The homepage and admin login page loaded through the local application server.

These tests verified application connectivity and page loading. They did not establish that the four separate PostgreSQL role accounts were being used by the application or that admin authentication was successfully completed.

7. Issues and Limitations

During the implementation of the database roles and privileges, several setup considerations and limitations were encountered.

7.1 Node.js and npm Environment Setup

Node.js and npm were installed and verified on the local Windows environment.

During testing, some Command Prompt windows could not recognize the node and npm commands because the Node.js installation directory was not available in the current PATH.

The issue was addressed by temporarily adding the Node.js installation directory to the PATH of the active Command Prompt window.

The following versions were successfully verified:

- Node.js: 26.13.0

- npm: 11.20.0

7.2 Backend Dependency Installation

The backend dependencies were installed using npm ci.

A total of 121 packages were installed successfully. The npm audit reported two moderate-severity vulnerabilities.

A warning was also displayed for the bcrypt installation script because it had not yet been approved through npm's install-script controls.

The bcrypt module was subsequently tested, and its hash function was confirmed to be available.

7.3 PostgreSQL Environment Setup

PostgreSQL 18.6 was used for the Activity 4 database.

The separate grp08_catering_db database was successfully prepared with the project's schema and seed data.

The backend database connection was configured using a local .env file. The file contains the local database credentials and session secrets and was not intended to be committed to the repository.

7.4 Application Integration

The Express.js application uses one shared PostgreSQL connection pool for its application routes.

The separate database roles created for Activity 4 were tested directly through PostgreSQL. However, these accounts have not yet been integrated into separate application database connections.

The current application configuration uses the PostgreSQL account specified in its local environment file rather than automatically switching database accounts based on the authenticated application user.

7.5 Customer Authentication

The existing customer login implementation retrieves customer information, including password_hash.

However, the customer database role intentionally does not have SELECT permission on that column.

Therefore, the existing customer login query is not compatible with direct use of the restricted customer account without a backend query adjustment or an alternative database authentication design.

7.6 Customer Record Isolation

The application filters customer reservation queries using the authenticated customer's ID.

However, the database permissions alone do not enforce row-level isolation. The customer database role can access permitted rows across the customer table and other granted tables.

Additional application-level authorization or PostgreSQL Row-Level Security would be needed to enforce customer-specific data isolation at the database level.

7.7 Application Authorization

The existing application uses admin session authentication for protected staff management routes.

Separate staff and customer PostgreSQL roles have not yet been connected to their respective application operations.

These limitations should be addressed before using the Activity 4 accounts as the application's production database accounts.

8. Evidence and Repository

The SQL script and supporting screenshots for the Group08 access-control implementation are organized in the following project directory:

https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group08

8.1 SQL Script

The SQL role and privilege configuration is saved at:

https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group08/sql

The script contains the commands used to create the group roles and login accounts, assign memberships, configure database and schema privileges, and apply table, column, and sequence permissions.

8.2 Evidence Screenshots

The supporting screenshots are organized in:

https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group08/evidence

The evidence covers the following categories:

1. Initial database structure and record counts.
2. Group role and login role configuration.
3. Role memberships.
4. Database and schema privileges.
5. Table and column privileges.
6. Sequence privileges.
7. Role security settings.
8. Customer account successful and denied access tests.
9. Staff account successful and denied access tests.
10. Auditor account successful and denied access tests.
11. Administrator account successful and denied access tests.
12. Database connection restrictions.
13. Final record verification.
14. Node.js database connection test.
15. Live application database connection test.

8.3 Application Testing Evidence

Additional application screenshots document the successful backend and frontend smoke tests.

These include:

- Successful Node.js database connection through npm run db:test.
- Successful Express backend startup using npm start.
- Successful response from the /api/db-test endpoint.
- Successfully loaded Catering Management System homepage.
- Successfully loaded admin login page.

These screenshots provide supplementary evidence that the application can connect to the isolated database and that its main pages load correctly.

9. SQL Scripts Summary

The grp08_roles_and_privileges.sql script contains the SQL commands used to configure database roles, assign privileges, and apply access restrictions for the Catering Business Management System.

The script covers the following operations:

- **Group Role Creation** – Created grp08_catering_admin_grp, grp08_catering_staff_grp, grp08_catering_customer_grp, and grp08_catering_readonly_grp as NOLOGIN roles.
- **Login Role Creation** – Created grp08_catering_admin, grp08_catering_staff, grp08_catering_customer, and grp08_catering_auditor as LOGIN roles.
- **Role Membership** – Assigned each login role to its corresponding group role.
- **Database Access** – Revoked CONNECT from PUBLIC on grp08_catering_db and granted CONNECT to the four intended group roles.
- **Schema Privileges** – Revoked CREATE from PUBLIC on the public schema and granted the required schema permissions to the group roles.
- **Administrative Privileges** – Granted SELECT, INSERT, UPDATE, and DELETE on all existing tables, together with ALL privileges on sequences, to the administrative group.
- **Staff Privileges** – Granted SELECT, INSERT, and UPDATE on permitted operational tables and required sequence privileges.
- **Customer Privileges** – Granted limited customer profile access, registration privileges, catalog access, and permitted reservation-related operations.
- **Auditor Privileges** – Granted SELECT only on inventory, menu_item, and package.
- **Access Restrictions** – Restricted access to sensitive customer password information, administrative records, and unauthorized database operations.
- **Privilege Verification** – Used PostgreSQL privilege-checking queries and direct login tests to verify permitted and denied operations.

The script follows the principle of least privilege by limiting each role to the permissions required for its assigned responsibilities. Passwords are configured separately and are not stored in the SQL script.

10. Verification Results

The database roles and privileges were checked using PostgreSQL queries and direct login tests.

10.1 Customer Account Verification

Account	Operation	Expected Result	Actual Result
grp08_catering_customer	Connect to database	Allowed	Successful
grp08_catering_customer	SELECT package and menu_item	Allowed	Successful
grp08_catering_customer	SELECT permitted profile columns	Allowed	Successful
grp08_catering_customer	INSERT customer record	Allowed	Successful
grp08_catering_customer	UPDATE permitted profile field	Allowed	Successful
grp08_catering_customer	DELETE reservation	Denied	Permission denied
grp08_catering_customer	SELECT password_hash	Denied	Permission denied
grp08_catering_customer	UPDATE restricted email field	Denied	Permission denied
grp08_catering_customer	Access admin table	Denied	Permission denied

10.2 Staff Account Verification

Account	Operation	Expected Result	Actual Result
grp08_catering_staff	Connect to database	Allowed	Successful
grp08_catering_staff	SELECT reservation records	Allowed	Successful
grp08_catering_staff	UPDATE reservation	Allowed	Successful
grp08_catering_staff	DELETE reservation	Denied	Permission denied
grp08_catering_staff	Access admin table	Denied	Permission denied
grp08_catering_staff	SELECT customer password_hash	Denied	Permission denied
grp08_catering_staff	SELECT permitted profile columns	Allowed	Successful

10.3 Auditor Account Verification

Account	Operation	Expected Result	Actual Result
grp08_catering_auditor	Connect to database	Allowed	Successful
grp08_catering_auditor	SELECT inventory	Allowed	Successful
grp08_catering_auditor	SELECT menu_item	Allowed	Successful
grp08_catering_auditor	SELECT package	Allowed	Successful
grp08_catering_auditor	SELECT reservations	Denied	Permission denied
grp08_catering_auditor	Access customer records	Denied	Permission denied
grp08_catering_auditor	INSERT inventory record	Denied	Permission denied

10.4 Administrative Account Verification

Account	Operation	Expected Result	Actual Result
grp08_catering_admin	Connect to database	Allowed	Successful
grp08_catering_admin	SELECT customer records	Allowed	Successful
grp08_catering_admin	SELECT reservation records	Allowed	Successful
grp08_catering_admin	SELECT admin records	Allowed	Successful
grp08_catering_admin	INSERT inventory record	Allowed	Successful
grp08_catering_admin	UPDATE inventory record	Allowed	Successful
grp08_catering_admin	DELETE inventory record	Allowed	Successful
grp08_catering_admin	CREATE table in public schema	Denied	Permission denied

The administrator's table-level privileges were tested separately from its schema CREATE permission.

10.5 Additional Privilege Verification

The following configurations were also checked:

- Group roles have the NOLOGIN attribute.
- Login roles have the LOGIN attribute.
- Login roles belong to their intended group roles.
- The four accounts do not have SUPERUSER, CREATEROLE, or CREATEDB privileges.
- The administrative group has the assigned table and sequence privileges.
- Staff and customer roles have restricted table and column permissions.
- The auditor has SELECT access only to the approved tables.
- The auditor has no sequence privileges.
- PUBLIC does not have CONNECT privileges on the isolated testing database.

10.6 Application Connection Verification

The Node.js backend was tested using the following commands:

Dependency Installation

- npm ci

Database Connection Test

- npm run db:test
- The database connection test returned:
    - Database connected successfully!
    - Connected database: grp08_catering_db

Backend Startup

- npm start
- The Express server successfully started at:
    - http://localhost:5000

Live API Test

- http://localhost:5000/api/db-test
- The endpoint successfully returned a database connection response identifying grp08_catering_db.
- The homepage and admin login page were also successfully loaded through the local Express server.
- These tests verified application connectivity and page loading. They did not establish that the four separate PostgreSQL role accounts were being used by the application or that admin authentication was successfully completed.

11. Issues and Fixes

The following issues and limitations were encountered during the Group08 database setup and access-control implementation.

Issue	Action Taken / Resolution	Status
Node.js and npm were not recognized in some Command Prompt windows.	Temporarily added C:\Program Files\nodejs to the active CMD PATH.	Resolved for the active session
Backend dependencies needed to be installed.	Executed npm ci in the backend directory.	Resolved
npm reported two moderate vulnerabilities.	Recorded the audit result and avoided npm audit fix --force to prevent unintended breaking changes.	Requires review
bcrypt installation script was not initially approved.	Checked the module directly and confirmed that its hash function was available.	Function verified
The backend initially could not recognize Node.js and npm in a new CMD window.	Verified the installation directory and restored access using a temporary PATH command.	Resolved for the active session
The backend needed a local database configuration.	Created a local .env file with the isolated database settings and private credentials.	Resolved
The backend needed to verify PostgreSQL connectivity.	Executed npm run db:test and confirmed the connected database.	Resolved
The frontend and backend needed supplementary testing.	Started the Express server and checked the homepage, admin login page, and live database API endpoint.	Passed

11.1 Additional Verification Completed

The following additional checks were completed after the database access-control implementation:

1. Verified the Node.js and npm versions.
2. Installed the backend dependencies successfully.
3. Verified that bcrypt was available.
4. Confirmed that the backend connected to grp08_catering_db.
5. Started the Express backend successfully.
6. Tested the live /api/db-test endpoint.
7. Confirmed that the frontend homepage loaded correctly.
8. Confirmed that the admin login page loaded correctly.

The application connection test used the local PostgreSQL credentials configured in .env. It was separate from the direct privilege tests performed using the four Activity 4 login accounts.

12. Feedback to the Owning Group

The following observations and recommendations were identified during the database setup and testing of the Catering Business Management System.

Area	Observation	Recommendation
Database Setup	The database schema and seed records were successfully imported into the isolated testing database.	Maintain clear database initialization instructions and backup procedures.
Database Privileges	Separate roles were configured for administration, staff, customers, and read-only access.	Continue using role-based permissions and avoid unnecessary privileges for application accounts.
Sensitive Information	Customer password hashes were restricted from staff and customer SELECT access.	Continue protecting sensitive customer information through column-level privileges and secure authentication queries.

Customer Access	Customer permissions allow access to permitted tables but do not independently enforce customer-specific row isolation.	Consider implementing PostgreSQL Row-Level Security or additional database-enforced restrictions.
Application Integration	The Express application currently uses a shared PostgreSQL connection pool.	Consider integrating separate database identities and appropriate authorization controls into application operations.
Backend Configuration	The backend successfully connected to the isolated PostgreSQL database using a local environment configuration.	Keep database credentials and session secrets outside the source repository.
Documentation	The role configuration and access-control tests were documented.	Keep the SQL script, evidence screenshots, and setup instructions organized in the project repository.

13. AI Usage Declaration

AI tools, including ChatGPT, were used as supplementary assistance during the Group08 database setup and access-control implementation.

The AI assistance was used for the following tasks:

- Understanding PostgreSQL roles, memberships, privileges, and access-control concepts.
- Reviewing SQL scripts for role creation and privilege assignment.
- Explaining PostgreSQL permission errors and troubleshooting database access issues.
- Providing guidance in configuring the Node.js and PostgreSQL environment.
- Preparing verification queries and identifying appropriate allowed and denied access tests.
- Organizing the progress documentation and explaining the technical test results.

The SQL implementation and database permissions were checked through actual PostgreSQL execution and direct login tests. The application connection was also verified through the Node.js database test script and the live Express API endpoint.

AI was used as a support tool. The group remains responsible for reviewing, validating, and understanding the final implementation and submitted documentation.

14. Conclusion

The Group08 Catering Business Management System database was successfully configured for Activity 4 access-control testing using PostgreSQL 18.6.

Four group roles and their corresponding login roles were created to separate administrative, staff, customer, and read-only responsibilities.

Privileges were assigned based on the principle of least privilege. The administrative role received the required table and sequence permissions, the staff and customer roles were limited to their permitted operations, and the auditor role was restricted to approved tables.

Verification included checking role memberships, database and schema privileges, table and column permissions, sequence privileges, and actual allowed and denied operations. The tests confirmed that the assigned permissions and access restrictions were working as intended.

The supplementary application testing also confirmed that the Node.js backend successfully connected to the isolated PostgreSQL database and that the frontend homepage and admin login page loaded correctly.

Some limitations remain regarding the integration of the separate PostgreSQL roles into the application's shared connection pool and database-enforced customer record isolation. These considerations were documented for future improvement.

Overall, the Activity 4 database role configuration, privilege assignment, and access-control verification were completed successfully. The SQL implementation and supporting evidence provide a record of the group's database administration work.